Dashboard Backend stack

Critère	Python + FastAPI	JavaScript + Node.js	Go + Gin	Laravel
Facilité d'apprentissage	5 - Python est simple à prendre en main et FastAPI introduit peu de concepts au départ.	4 - Node.js est accessible, mais JavaScript et l'asynchronisme peuvent devenir complexes.	4 - Go a une syntaxe simple, mais le typage statique, les pointeurs et la concurrence demandent un apprentissage.	4 - Laravel est accessible, mais nécessite de comprendre PHP et les conventions du framework.
Facilité d'utilisation	5 - FastAPI intègre naturellement le routage, la validation et la documentation API.	5 - Node.js est très flexible grâce à son vaste écosystème.	4 - Gin est léger et simple, mais fournit volontairement peu de fonctionnalités intégrées.	5 - Laravel fournit de nombreux outils prêts à l'emploi pour le backend.
Vitesse de développement	5 - FastAPI permet de développer rapidement des API avec peu de code et une validation automatique.	5 - Node.js permet une itération rapide grâce à npm et à son grand nombre de bibliothèques.	4 - Go est rapide à développer une fois maîtrisé, mais privilégie un code explicite et parfois plus verbeux.	5 - Laravel accélère fortement le développement grâce à ses nombreuses fonctionnalités intégrées.
Typage	2 - Python est dynamiquement typé, même si les annotations améliorent la vérification du code.	2 - JavaScript est dynamiquement typé, sauf si TypeScript est ajouté.	5 - Go impose un typage statique fort vérifié à la compilation.	3 - PHP moderne propose du typage, mais reste plus permissif que Go.
Sécurité	4 - FastAPI offre une bonne validation des requêtes mais nécessite de surveiller les dépendances Python.	3 - Node.js dispose de bons outils de sécurité mais son grand nombre de dépendances augmente les risques.	5 - Go combine typage statique, faible surface de dépendances et sécurité mémoire.	4 - Laravel intègre plusieurs mécanismes de sécurité comme la protection CSRF et le hachage des mots de passe.
Simplicité	5 - FastAPI permet de créer une API fonctionnelle avec très peu de code.	4 - Node.js est simple à exécuter mais son écosystème peut rendre l'architecture plus complexe.	5 - Go repose sur un langage réduit, explicite et avec peu d'abstractions.	4 - Laravel est pratique mais reste un framework riche avec de nombreux concepts.
Maintenabilité	4 - FastAPI se maintient bien avec une bonne architecture, malgré le typage dynamique de Python.	3 - Node.js peut être maintenable avec une architecture stricte, surtout avec TypeScript.	5 - Le typage statique et la simplicité de Go facilitent la maintenance des gros projets.	4 - Les conventions de Laravel facilitent la maintenance, même si ses abstractions peuvent devenir complexes.
Efficacité mémoire	2 - Python consomme relativement beaucoup de mémoire à cause du runtime et de ses objets.	3 - Node.js est optimisé mais conserve un coût mémoire lié au moteur V8.	5 - Go utilise peu de mémoire grâce à son runtime léger et ses représentations natives.	3 - Laravel a une consommation mémoire correcte mais supérieure à Go.
Performances	4 - FastAPI est performant pour les API, mais reste généralement derrière Go sur les traitements lourds.	4 - Node.js est performant pour les charges dominées par les entrées-sorties.	5 - Go offre d'excellentes performances grâce à la compilation native et à son faible overhead.	3 - Laravel est performant pour le web classique mais subit le coût de ses abstractions.
Concurrence	3 - Python gère bien l'I/O asynchrone mais reste limité pour le parallélisme CPU.	4 - Node.js gère très bien de nombreuses opérations I/O grâce à son modèle événementiel.	5 - Les goroutines et les channels rendent la concurrence simple et efficace en Go.	3 - PHP convient bien aux requêtes web classiques mais moins aux services très concurrents et persistants.
Scalabilité	4 - FastAPI se scale bien horizontalement mais peut consommer davantage de ressources.	4 - Node.js se scale bien pour les services distribués et orientés I/O.	5 - Go est très adapté à la montée en charge grâce à ses performances et sa faible consommation de ressources.	4 - Laravel peut bien monter en charge avec une infrastructure adaptée et du caching.
Écosystème	5 - Python possède un écosystème immense couvrant le web, la data, l'IA et l'automatisation.	5 - npm est l'un des plus grands écosystèmes de packages au monde.	4 - L'écosystème Go est plus réduit mais très solide pour le backend, le cloud et le réseau.	4 - Laravel bénéficie d'un écosystème PHP mature et très complet.

Critère	Python + FastAPI	JavaScript + Node.js	Go + Gin	Laravel
Communauté	5 - Python et FastAPI disposent d'une communauté importante et active.	5 - JavaScript possède l'une des plus grandes communautés de développeurs.	4 - La communauté Go est plus petite mais très forte sur le backend et l'infrastructure.	4 - PHP et Laravel disposent d'une communauté large, mature et active.
Documentation	5 - FastAPI propose une documentation claire et génère automatiquement la documentation OpenAPI.	4 - Node.js est bien documenté, même si son écosystème est fragmenté entre de nombreux outils.	4 - Go dispose d'une excellente documentation officielle.	5 - Laravel est reconnu pour sa documentation complète et accessible.
Outils de débogage	5 - Python dispose de nombreux outils de débogage, profilage et analyse mémoire.	5 - Node.js bénéficie notamment des outils V8 et Chrome DevTools.	5 - Go propose Delve, pprof, tracing et un détecteur de data races intégré.	4 - Laravel et PHP disposent de bons outils comme Xdebug, mais l'ensemble est moins intégré.
Tests	5 - Python bénéficie de pytest et d'un écosystème de tests très complet.	5 - Node.js dispose de nombreux outils de test comme Jest, Vitest et le runner natif.	5 - Go intègre directement les outils de test dans sa bibliothèque standard.	5 - Laravel fournit une couche de test très complète au-dessus des outils PHP.
Fonctionnalités backend intégrées	4 - FastAPI fournit d'excellentes briques pour les API mais n'est pas un framework applicatif complet.	3 - Node.js fournit peu de fonctionnalités backend haut niveau sans bibliothèques supplémentaires.	3 - Gin se concentre surtout sur le routage HTTP et les middlewares.	5 - Laravel intègre ORM, migrations, authentification, files de jobs, cache et événements.
Flexibilité d'architecture	5 - FastAPI impose peu de contraintes et laisse une grande liberté d'organisation.	5 - Node.js laisse au développeur un très grand choix d'architecture et de bibliothèques.	5 - Gin impose peu de conventions et laisse un contrôle important sur l'architecture.	3 - Laravel encourage fortement sa propre architecture et ses conventions.
Gestion des dépendances	4 - Python propose plusieurs outils efficaces, même si son packaging peut parfois être complexe.	3 - npm est puissant mais peut générer des arbres de dépendances très importants.	5 - Les modules Go sont simples, reproductibles et intégrés à l'outillage officiel.	4 - Composer est mature, fiable et largement utilisé dans l'écosystème PHP.
Déploiement	3 - FastAPI nécessite un environnement Python et généralement un serveur ASGI ou un conteneur.	4 - Node.js se déploie facilement avec un runtime ou un conteneur.	5 - Go compile en un binaire autonome, ce qui simplifie fortement le déploiement.	3 - Laravel nécessite un environnement PHP et davantage de configuration serveur.
Développement d'API	5 - FastAPI intègre directement routage, validation, sérialisation et OpenAPI.	5 - Node.js offre un grand choix de frameworks et de bibliothèques pour créer des API.	5 - Gin fournit un routage et des middlewares simples, rapides et peu coûteux.	5 - Laravel intègre routage, validation, authentification et ORM pour développer des API complètes.
Pertinence à long terme pour de gros services	4 - FastAPI reste très capable mais le typage dynamique et la consommation de ressources peuvent devenir des limites.	4 - Node.js est éprouvé à grande échelle mais bénéficie fortement de TypeScript et de conventions strictes.	5 - Go combine performance, typage statique, simplicité et efficacité opérationnelle pour les gros services.	4 - Laravel reste très adapté aux applications métier et web structurées.
TOTAL	**93**	**89**	**102**	**88**

Notation sur 5 : 1 = peu favorable, 5 = très favorable.

Nous avons choisi Go avec Gin pour le backend car cette stack offre un bon compromis entre simplicité, performances, sécurité et maintenabilité. Go bénéficie d'un typage statique fort, d'une syntaxe explicite et d'une gestion efficace de la concurrence, tandis que Gin fournit un framework HTTP léger qui laisse une grande liberté dans l'organisation de l'architecture. La compilation en un binaire autonome simplifie également le déploiement et la dockerisation de l'application.

Une autre stack que nous aurions pu choisir est Python avec FastAPI. Elle obtient également un très bon score dans notre benchmark grâce à sa facilité d'apprentissage, sa rapidité de développement et ses outils intégrés pour créer des API, notamment la validation des données et la génération automatique de la documentation OpenAPI. Cette stack aurait permis de développer rapidement le backend du dashboard avec peu de code.